

Router → Service → Repository Pattern

1. Konteks & Motivasi

Setelah W11D01 menjelaskan *apa* yang boleh dan tidak boleh dilakukan tiap layer secara konseptual, W11D02 adalah implementasinya — bagaimana tiga layer itu benar-benar ditulis dalam kode nyata yang bisa dijalankan. Ini bukan soal mengikuti konvensi estetika. Ini soal membuat kode yang bisa diuji, diubah, dan di-maintain tanpa takut merusak bagian lain.

2. Analogi — Rumah Sakit

Bayangkan sebuah rumah sakit yang sehat:

- **Resepsionis (Router)** — terima pasien, cek formulir, arahkan ke ruangan. Tidak mendiagnosis.
- **Dokter (Service)** — analisis kondisi, buat keputusan medis. Tidak ambil sampel darah sendiri.
- **Lab & Arsip (Repository)** — simpan dan ambil data. Tidak membuat keputusan medis apapun.

Kalau resepsionis mulai mendiagnosis → chaos. Kalau dokter turun ke lab sendiri → tidak scalable. Setiap layer punya *satu* tanggung jawab, berkomunikasi lewat kontrak yang jelas.

3. Struktur Folder Week 11

```
week-11/app/
├── model/
│   ├── __init__.py
│   ├── loader.py           ← load_model() – infrastructure, bukan service
├── routers/
│   ├── predict.py         ← hanya routing + error translate + Depends()
├── schemas/
│   ├── prediction.py      ← Pydantic schema, tidak perlu diubah dari W08
├── services/
│   ├── model_service.py   ← hanya predict(), tanpa load logic
├── repositories/          ← kosong dulu, diisi saat database masuk (W13)
├── main.py
├── config.py
└── database.py
```

Catatan penting: repositories/ sengaja dikosongkan. Menambahkan ScoreRepository sekarang tanpa database dependency nyata adalah *over-engineering* — layer yang dibuat bukan karena kebutuhan, tapi karena terlihat benar secara konsep. Itu pelanggaran prinsip yang sama.

4. Aturan Per Layer — Tabel Referensi

	Router	Service	Repository
Import FastAPI?	✓ Ya	✗ Tidak	✗ Tidak

Import HTTPException?	✓ Ya	✗ Tidak	✗ Tidak
Business logic?	✗ Tidak	✓ Ya	✗ Tidak
Query data langsung?	✗ Tidak	✗ Tidak	✓ Ya
Exception yang dilempar	HTTPException	ValueError	LookupError / None

5. Alur Data — Request ke Response

Request masuk → Router (validasi format) → Service (business logic) → Repository (ambil data) → Service (proses hasil) → Router (translate ke HTTP response) → Response keluar. **Tidak ada layer yang boleh lompat.** Router tidak boleh langsung ke Repository.

6. Exception Language Per Layer

Exception punya 'bahasa' yang berbeda tiap layer — ini bukan pilihan style, ini kontrak arsitektur:

Layer	Exception yang Dilempar	Ditangkap Oleh	Ditranslasi Ke
Repository	LookupError	Service	ValueError atau diteruskan
Service	ValueError / RuntimeError	Router	HTTPException(422/503)
Router	HTTPException	FastAPI framework	HTTP Response

■ ■ **Anti-pattern yang harus dihindari:** Menaruh `from fastapi import HTTPException` di dalam service layer. Ini disebut *framework leak* — service menjadi terikat ke FastAPI. Jika besok logic yang sama di-expose via CLI atau gRPC, service itu akan rusak.

7. Implementasi — loader.py

Infrastructure murni. Satu fungsi, satu return. Tidak ada business logic, tidak ada class yang tidak perlu.

```
# app/model/loader.py
import joblib
import logging

logger = logging.getLogger(__name__)

def load_model(model_path: str):
    """Load model bundle dari disk. Return pipeline langsung."""
    try:
        bundle = joblib.load(model_path)
        logger.info(f"Model loaded from {model_path}")
        return bundle
    except Exception as e:
        logger.error(f"Failed to load model: {e}")
        raise # bukan 'raise e' — pertahankan original traceback
```

Kenapa function, bukan class? Class worth it saat ada state yang perlu dijaga antar method call, banyak instance serupa perlu dibuat, atau ada pewarisan. `load_model()` tidak punya kondisi itu — dipanggil sekali, selesai.

raise vs raise e: Selalu pakai raise tanpa argumen di dalam except block. raise e membuat traceback baru yang memotong informasi — kamu kehilangan jejak di mana error sebenarnya terjadi.

8. Implementasi — model_service.py

Service layer. Tidak ada FastAPI import. Tidak ada HTTPException. Business logic ada di sini — transformasi data, prediksi, interpretasi hasil.

```
# app/services/model_service.py
import numpy as np
import logging

logger = logging.getLogger(__name__)

class ModelService:
    def __init__(self, model, version: str):
        self.model = model      # model di-inject dari luar (DI)
        self.version = version

    def predict(self, features: list[float]) -> dict:
        if self.model is None:
            raise RuntimeError("Model pipeline is not available.")

        X = np.array(features).reshape(1, -1) # reshape untuk sklearn
        prediction = self.model.predict(X)[0]

        probability = None
        if hasattr(self.model, 'predict_proba'):
            proba = self.model.predict_proba(X)[0]
            probability = float(max(proba))

        return {
            "prediction": int(prediction) if isinstance(prediction, (np.integer, int))
                           else float(prediction),
            "probability": probability,
            "model_version": self.version
        }
```

9. Implementasi — predict.py

Router layer. Tiga except block terpisah — bukan satu except Exception. Setiap exception type ditranslasi ke HTTP status code yang jujur.

```
# app/routers/predict.py
from fastapi import APIRouter, HTTPException, Request, Depends
from schemas.prediction import PredictionRequest, PredictionResponse
import logging

logger = logging.getLogger(__name__)
router = APIRouter(prefix='/api/v1', tags=['prediction'])

def get_model_service(request: Request):
    return request.app.state.model_service

@router.post('/predict', response_model=PredictionResponse)
async def predict(
```

```

    payload: PredictionRequest,
    model_service = Depends(get_model_service)
):
    try:
        result = model_service.predict(payload.to_feature_list())
        return result
    except ValueError as e:
        logger.warning(f"Validation error: {e}")
        raise HTTPException(status_code=422, detail=str(e))
    except RuntimeError as e:
        logger.error(f"Model unavailable: {e}")
        raise HTTPException(status_code=503, detail=str(e))
    except Exception as e:
        logger.error(f"Unexpected error: {e}")
        raise HTTPException(status_code=500, detail="Internal server error")

```

Kenapa tiga except, bukan satu? Satu except Exception memukul rata semua error jadi 500. Ini berbohong ke client: 422 = kesalahan user, 503 = sistem tidak siap, 500 = bug tak terduga. Tiga situasi berbeda → tiga status code berbeda.

10. Implementasi — schemas/prediction.py

Schema tahu strukturnya sendiri — logika transformasi struktur ada di schema, bukan di router. Router tidak perlu tahu field apa saja yang ada, cukup panggil `to_feature_list()`.

```

# app/schemas/prediction.py
from pydantic import BaseModel, Field

class PredictionRequest(BaseModel):
    feature_1: float = Field(..., description='Informative synthetic feature')
    feature_2: float = Field(..., description='Informative synthetic feature')
    feature_3: float = Field(..., description='Informative synthetic feature')
    feature_4: float = Field(..., description='Redundant synthetic feature')

    model_config = {
        "json_schema_extra": {
            "examples": [{"feature_1": 1.5, "feature_2": -0.5,
                           "feature_3": 2.1, "feature_4": 0.8}]
        }
    }

    def to_feature_list(self) -> list[float]:
        return [self.feature_1, self.feature_2,
                self.feature_3, self.feature_4]

class PredictionResponse(BaseModel):
    prediction: int | float
    probability: float | None = None
    model_version: str

```

Konvensi urutan Pydantic: fields dulu → model_config → methods. Method yang dideklarasikan sebelum field membingungkan pembaca — mereka harus scroll ke bawah untuk tahu field apa saja yang ada sebelum memahami method-nya.

11. Pelanggaran Umum — Cara Mengenalinya

Kode Bermasalah	Layer	Masalah	Solusi
from fastapi import HTTPException di service	Service	Framework leak	Lempar ValueError, router yang translate
db.query(User) langsung di service	Service	Coupled ke ORM	Buat Repository, inject ke service
score = model.predict(X)[0] di router	Router	Business logic bocor ke router	Pindahkan ke service layer
BackgroundTasks sebagai param service	Service	FastAPI construct masuk service	Tangani di router
raise HTTPException di repository	Repo	Layer tahu soal HTTP	Lempar LookupError

12. Rangkuman — Yang Harus Diingat

- **loader.py = function, bukan class.** Class only when: state antar method, banyak instance, atau inheritance.
- **raise, bukan raise e.** raise tanpa argumen pertahankan original traceback.
- **Service tidak boleh import FastAPI apapun.** Tidak ada HTTPException, tidak ada BackgroundTasks.
- **Exception punya bahasa per layer:** ValueError di service, HTTPException di router, LookupError di repo.
- **Tiga except terpisah** di router — bukan satu except Exception. Status code yang jujur ke client.
- **to_feature_list() ada di schema** karena schema tahu strukturnya sendiri. Router tidak perlu tahu detail field.
- **repositories/ kosong dulu.** Diisi saat ada database dependency nyata (W13). Bukan over-engineering.
- **Konvensi urutan Pydantic:** fields → model_config → methods.